# Sort the Vocabulary: Prefix-Ordered Token Classes Make Two-Level Language-Model Heads Cheap and Faithful

Lucas Tinoco*

October 2026

**Abstract**

In small language models with large subword vocabularies, the output head is a large share of the per-token cost: 21–63% of the weight bytes read per generated token in Gemma 3 270M, Qwen3-0.6B and Llama 3.2 1B. A two-level (class-factored) softmax cuts that cost by $\sqrt{V}$-fold, and has returned in 2025–26 in speculative-decoding drafters and drop-in head replacements; what remains open is *how to form the classes*. We show that the simplest possible rule—sort the subword vocabulary by spelling and cut it into equal blocks—gives the most faithful factored head among the criteria in current use. Swapping the head of five frozen open models (Gemma 3 270M/1B, Gemma 4 E2B, Qwen3-0.6B, gpt-oss-20b; three vendors, four tokenizers), prefix-ordered classes have the lowest KL divergence to the original model in four of five (tied with semantic $k$-means on the fifth), and 27–33% lower KL than the token ordering Google ships with its Gemma 4 drafters, at equal cost, including on Gemma 4 itself. Trained from scratch, prefix classes beat frequency, merge-order, random and semantic classes in recurrent, MLP and Transformer models, in Portuguese and English, with the gap shrinking as the vocabulary grows. A per-decision and word-start/continuation decomposition explains why: the first decision follows the generation direction, and the gain concentrates on tokens that continue a word—which also explains why alphabetical classes lost at the whole-word level (Jiang et al., 2017). On a CPU at batch size one, the factored head makes Gemma 3 270M generate 2.4× faster end to end (2.1× with AVX2 only).

## 1 Introduction

The output projection of a language model maps a hidden state $h \in \mathbb{R}^d$ to logits over a vocabulary of size $V$. Vocabularies have grown (128k in Llama 3, 152k in Qwen3, 201k in gpt-oss, 262k in Gemma) while the models people run locally have stayed small, so in small models the head is no longer a rounding error: it is 63% of the bytes read per token in Gemma 3 270M (Table 4). At batch size one—a model on a laptop or phone, or a draft model in speculative decoding—generation is memory-bound, and those bytes are time and energy.

Class-factored softmax [1, 2] is the classical remedy: predict a class, then a token inside the class, each with a small softmax; with $C \approx \sqrt{V}$ classes of $m \approx \sqrt{V}$ tokens the head reads $(C+m)d$ instead of $Vd$ weights. It fell out of use on GPUs with large batches, where the full head is cheap, and because it costs quality. It is back: Gemma 4's drafters score 2,048 token clusters and expand the top 32 [10]; FlashHead replaces heads with balanced spherical $k$-means clusters and multi-probe

*Independent researcher. Implementation, experiments and drafting were carried out with the assistance of an AI coding agent (Claude); all results were run and checked on the author's machine. Code and raw results: `https://github.com/lucasrgt/prefix-head`.

retrieval [9]; CORAL factors a drafter head into equal groups [11]; FR-Spec and VocabTrim restrict drafter heads by frequency [12, 13]. All of these group tokens by embedding similarity, frequency, or token id.

We ask which grouping makes the factored head most faithful, at equal cost, and find that a rule needing no data and no model—*sort the vocabulary by spelling, cut into equal blocks*—wins. Our contributions:

- **Retrofit evidence on five frozen open models** (Section 3): prefix classes give the lowest KL to the original head in four of five models (a tie in the fifth), and beat the ordering shipped with Gemma 4 by 27–33% in KL at equal cost.
- **From-scratch evidence** across MLP, recurrent and Transformer models, two languages and vocabularies of 8k–100k (Section 4), including the honest finding that the gap shrinks with vocabulary size.
- **A mechanism** (Section 5): the class decision becomes easy because it follows the generation direction; the gain is 4–6× larger on word-continuation tokens. This reconciles our result with the opposite whole-word finding of Jiang et al. [5].
- **End-to-end cost** on CPU and a consumer GPU at batch one (Section 6).

## 2 Method

**Two-level head.** Given a partition of the vocabulary into classes $c(t)$ with members $M_c$, the head computes

$$\log p(t \mid h) = \log \mathrm{softmax}(Ah + b)_{c(t)} + \log \mathrm{softmax}\big(W_{M_{c(t)}}\, h\big)_t,$$

where $A \in \mathbb{R}^{C \times d}$ predicts the class and $W_{M_c}$ are the output rows of the class members. The factorization is exact: no token is ever excluded. Sampling draws a class, then a token. At inference the head reads $C \cdot d$ class weights and one block of $m \cdot d$ token weights.

**Groupings compared.** *Prefix*: sort tokens by their byte spelling, cut into equal blocks of $m$. *Frequency*: sort by training frequency (the criterion of adaptive softmax and frequency-trimmed drafters). *Token id / merge order*: the tokenizer's own order (CORAL-style groups). *Random*. *Semantic*: balanced $k$-means on the output embeddings (as in FlashHead and, by our inspection, Gemma 4). *Gemma 4*: the `token_ordering` buffer released with the Gemma 4 E4B drafter, cut into its 2,048 blocks of 128.

**Retrofitting a frozen model.** We keep the model and its output rows frozen and train only $A, b$. In the *self* setting the class predictor is trained to reproduce the model's own class marginals $p(c \mid h) = \sum_{t \in M_c} p_{\text{full}}(t \mid h)$ on training text (no labels, no other model), so its deviation from the full head is the pure cost of factoring. We report $\mathrm{KL}(p_{\text{full}} \,\|\, p_{\text{2-level}})$ in bits per character on held-out text; because $p(t \mid c, h)$ is exact, this equals the class-level KL. In the *adapt* setting both the full head and the two-level head (class predictor and blocks) are fine-tuned on the same text with the same budget.

Table 1: Fidelity of the factored head to the original model: $\mathrm{KL}(p_{\text{full}} \| p_{\text{2-level}})$ in bits/char on held-out text (lower is better). Prefix, semantic and the 128-block columns: mean ± s.d. over 3 seeds (2 for semantic on Gemma 3 270M); others: 1 seed.

| model (vocab, classes) | prefix | semantic | frequency | random | token id |
|---|---|---|---|---|---|
| Gemma 3 270M-it (262k, 512) | **0.1368**±.0001 | 0.1621±.0045 | – | – | – |
| Gemma 3 1B-pt (262k, 512) | 0.0867±.0000 | **0.0854**±.0014 | 0.133 | 0.192 | 0.214 |
| Gemma 4 E2B-it (262k, 512) | **0.1838**±.0001 | 0.2738±.0024 | 0.271 | 0.329 | 0.364 |
| Qwen3-0.6B (152k, 390) | **0.0958**±.0001 | 0.1953±.0026 | 0.133 | 0.177 | 0.200 |
| gpt-oss-20b (201k, 448) | **0.0197**±.0001 | 0.0337±.0019 | 0.024 | 0.042 | 0.037 |

*equal cost, 2,048 classes of 128: prefix vs. the ordering shipped with Gemma 4 (3 seeds)*

| model | prefix | Gemma 4 | prefix reduction |
|---|---|---|---|
| Gemma 3 270M-it | **0.1158**±.0001 | 0.1593±.0003 | 27% |
| Gemma 3 1B-pt | **0.0724**±.0001 | 0.1086±.0001 | 33% |
| Gemma 4 E2B-it | **0.1656**±.0005 | 0.2414±.0005 | 31% |

## 3 Retrofitting frozen models

**Setup.** Hidden states after the final norm are extracted with llama.cpp from 14 public-domain Portuguese novels (train 650–700k positions; test 714k characters, disjoint). Models: Gemma 3 270M-it and 1B-pt, Gemma 4 E2B-it (logit soft-capping 30 applied), Qwen3-0.6B, gpt-oss-20b (o200k tokenizer). Blocks of $m = \lceil \sqrt{V} \rceil$ (head cost cut 224–256×), plus $m = 128$ to match Gemma 4's ordering (120×).

**Results.** Table 1. Prefix is the most faithful grouping in four of five models; on Gemma 3 1B semantic $k$-means is marginally better (0.0854 vs. 0.0867, within one s.d. of its seed spread), but semantic classes are is worse than frequency on Qwen3 and Gemma 4. At equal cost prefix blocks deviate 27–33% less than Gemma 4's own ordering, including on Gemma 4.

**Bits per character, and a calibration caveat.** On base models the factored head costs a little: Gemma 3 1B-pt 1.561→1.615 bpc (+3.5%; 1.600 with 128-blocks), Qwen3-0.6B 1.918→1.942 (+1.2%), gpt-oss-20b 2.685→2.697 (+0.5%). On the instruction-tuned Gemmas the self-trained factored heads score *below* the full head (Gemma 4 E2B-it: 2.604→2.475), and the bpc ranking of groupings even inverts; the smoother class predictor partly corrects the instruction-tuned model's miscalibration on this out-of-domain text. We therefore use KL to the original model, not bpc, as the primary retrofit metric.

**Equal-budget adaptation.** Fine-tuning both heads on the same text (Gemma 3 270M, Qwen3-0.6B): full 1.794 / 1.612 bpc; prefix 1.819 / 1.661 (+1.3% / +3.0%); semantic 1.842 / 1.751; frequency 1.896 / 1.724; random 1.872 / 1.723; token id 1.916 / 1.759. With 128-blocks on Gemma 3 270M: prefix +0.8%, Gemma 4 ordering +3.0%.

## 4 Training from scratch

We train small models on the same 14 novels with BPE vocabularies, the factored head with $C \approx m \approx \sqrt{V}$, and compare groupings at identical cost (raw runs: `results/alpha.jsonl` in the repository).

Table 2: From scratch, test bpc (lower is better). Means over seeds where given.

| model | prefix | semantic | merge order | frequency | random |
|---|---|---|---|---|---|
| MLP, BPE 8k (3 seeds) | **2.208** | 2.258 | 2.240–2.248 | 2.240–2.248 | – |
| Recurrent, BPE 8k (3 seeds) | **2.063** | 2.143 | 2.139[1] | 2.134[1] | 2.158[2] |
| Recurrent, English, BPE 8k | **1.762**[1] | – | 1.813[1] | – | – |
| MLP, English, BPE 8k (2 seeds) | **1.889** | – | 1.913 | 1.913 | – |

[1] one seed; [2] two seeds.

Table 3: Decomposition of test bits/char (recurrent model, BPE 8k): class decision + within-class decision, split by tokens that start a word and tokens that continue one.

| grouping | word start (527k chars) | continuation (189k chars) |
|---|---|---|
| prefix | 1.151 + 0.818 = **1.968** | 1.278 + 1.043 = **2.320** |
| merge order | 1.172 + 0.836 = 2.007 (+1.9%) | 1.641 + 0.863 = 2.504 (+7.9%) |
| semantic | 1.266 + 0.730 = 1.997 (+1.4%) | 1.953 + 0.578 = 2.531 (+9.1%) |
| frequency | 1.027 + 0.975 = 2.002 (+1.7%) | 1.542 + 0.959 = 2.501 (+7.8%) |
| second character | 1.220 + 0.773 = 1.994 (+1.3%) | 1.468 + 0.916 = 2.384 (+2.7%) |

**Vocabulary size.** The advantage over frequency shrinks as the vocabulary grows: 3.5% (8k), 2.2% (32k), 1.1% (100k; merge order 3.8%, 2.3%, 1.4%). Larger vocabularies contain more whole-word tokens, and fewer word continuations, which is where prefix classes help (Section 5). The retrofit gaps on 152k–262k vocabularies are larger because a frozen body cannot adapt to the grouping.

**Transformer.** A 2-layer Transformer with the prefix head reaches 2.063 bpc against 2.054 with the full 8k head (+0.45%) at 4.2× fewer MACs per character.

## 5 Why prefix classes work

Two hypotheses could explain the effect. *Separability*—semantic classes force a hard choice between synonyms inside a class—is refuted: the within-class decision is *easier* for semantic classes (0.69 vs. 0.88 bits/char overall). *Direction* is supported: with prefix classes the first decision asks how the next piece of text begins, which the context predicts well, especially when a word is in progress. Sorting by the second character instead gives an intermediate result. Table 3 localizes the gain: 4–6× larger on continuation tokens, and almost entirely in the class decision (1.28 vs. 1.54–1.95). With whole-word vocabularies every token starts a word, where the effect is small—consistent with Jiang et al. [5], where alphabetical classes lost to frequency and Brown clusters.

## 6 Cost

**GPU, head only.** On an RTX 4060 (fp16, CUDA graphs), scoring or sampling one token with the two-level head is 31–46× faster than the full head at batch one for the Llama 3.2 1B, Qwen3-0.6B and Gemma 3 270M head shapes, 20–31× at batch 8, and 2–10× at batch 512 with a naive per-row block gather.

Table 4: Output head as a share of the weight bytes read per generated token (Q8_0 GGUF, tied embeddings read as the head).

| model | vocab × $d$ | head params | head share of bytes/token |
|---|---|---|---|
| Gemma 3 270M | 262k × 640 | 168M of 268M | 63% |
| Gemma 3 1B | 262k × 1152 | 302M of 1.0B | 30% |
| Qwen3-0.6B | 152k × 1024 | 156M of 596M | 26% |
| Llama 3.2 1B | 128k × 2048 | 263M of 1.24B | 21% |
| SmolLM2-135M | 49k × 576 | 28M of 135M | 21% |

Table 5: Gemma 3 270M (Q8_0) end to end on CPU, batch one, tokens/s (mean of 3 runs of 512 tokens).

| ISA, threads | full head | prefix head (512×512) | speedup | full-head share of time |
|---|---|---|---|---|
| AVX-512, 8 | 131.9 | 321.2 | 2.4× | 56% |
| AVX-512, 1 | 75.5 | 183.5 | 2.4× | 57% |
| AVX2, 8 | 119.2 | 256.0 | 2.1× | 52% |
| AVX2, 1 | 58.2 | 120.5 | 2.1× | 52% |

**CPU, end to end.** We wrote a minimal Gemma 3 runtime in Rust (Q8_0 weights, AVX-512/AVX2, persistent thread pool) that matches llama.cpp to 0.3% in bpc over 700 tokens, and swapped only the head. Table 5 reports tokens/s on an idle Ryzen 7 7700 (Zen 4, 32 MB L3, dual-channel DDR5), 512 tokens of test text scored one at a time, mean of three runs. The full head takes 51–57% of each token's time; the prefix head (512 classes of 512) takes 1–2%. Whole-model throughput rises 2.4× with AVX-512 and 2.1× with AVX2, at 1 or 8 threads; with 2,048 classes of 128 the gain is 2.4× as well. Package energy per token (RAPL, total, not idle-subtracted, because the spinning thread pool keeps the package busy) falls by the same factor (0.58 to 0.24 J/token at 8 threads). Bits per character on these tokens: full 2.489, prefix head 2.435 (instruction-tuned model; see the calibration caveat above).

## 7 Related work

Class-based and hierarchical softmax [1, 3, 2]; adaptive softmax [4]; tree-based hierarchical softmax with alphabetical, Huffman and Brown trees at the word level [5]; prefix-tree lexicons for speech decoding search [6]. Screening and approximate heads: SVD-softmax [7], L2S [8], FlashHead [9]. Drafter heads: Gemma 4 [10], CORAL [11], FR-Spec [12], VocabTrim [13]. Training-memory reduction of large heads without changing inference cost: Cut Cross-Entropy [14]. Semantic hierarchies for decoding: HDLM and TDLM. To our knowledge no prior work orders a subword vocabulary by spelling to form output classes.

## 8 A frugal recurrent body (secondary study)

For CPU deployment we pair the head with a diagonal complex linear recurrence $h_t = (1 - z_t)\, e^{i\theta} h_{t-1} + z_t c_t$ with input-only gates and a learned, data-independent rotation. We arrived at it independently, but it is the HGRU recurrence of HGRN [17]; our differences are a chrono-style log-uniform initialization of gate biases and stateful truncated-BPTT training. At tiny scale (2.2M

parameters, 2 layers, 51 complex channels per layer) it matches a 2-layer Transformer with the same head (2.063 vs. 2.063 bpc) at 3.6× fewer MACs per character and an 816-byte inference state instead of a growing 128 KB KV cache, and beats our Mamba-2-style baseline (2.078 bpc at 1.6× the MACs). Removing the rotation ($\theta = 0$, so the real and imaginary halves become $2 \times 51$ independent real channels in the same 816 bytes) costs 0.8%: 2.079 bpc (2 seeds: 2.0799, 2.0789) against 2.063 (3 seeds: 2.0614, 2.0665, 2.0608). At this scale the rotation helps, unlike the findings of HGRN2 and Griffin at larger scale with expanded real states. We also tried a synthetic associative-recall task (2–32 key–value pairs per line, 2M characters, 30 epochs): neither the recurrent core with or without rotation nor a same-width Transformer learned it within that budget (recurrent: 3–5% accuracy on recalled characters, chance 3.8%; Transformer: 5–7%, declining with the number of pairs), so this probe is inconclusive and the known recall weakness of fixed-size states [15, 16] remains untested here.

## 9 Limitations

One text domain (19th-century public-domain literature) for all retrofit and most scratch results, plus English for replication. Retrofit models up to 21B parameters (3.6B active); scratch models are tiny. We evaluate likelihood and KL, not downstream tasks, and not speculative-decoding acceptance rates, which is how Gemma 4 uses its ordering (top-32 clusters, approximate). The factored head does not reduce the stored model size, only the bytes read per token. Gains at large batch on GPUs are modest.

## Acknowledgments

Thanks to Gabriel for many ideas along the way.

## References

[1] J. Goodman. Classes for fast maximum entropy training. ICASSP, 2001.

[2] T. Mikolov, S. Kombrink, L. Burget, J. Černocký, S. Khudanpur. Extensions of recurrent neural network language model. ICASSP, 2011.

[3] F. Morin, Y. Bengio. Hierarchical probabilistic neural network language model. AISTATS, 2005.

[4] E. Grave, A. Joulin, M. Cissé, D. Grangier, H. Jégou. Efficient softmax approximation for GPUs. ICML, 2017.

[5] N. Jiang, W. Rong, M. Gao, Y. Shen, Z. Xiong. Exploration of tree-based hierarchical softmax for recurrent language models. IJCAI, 2017.

[6] H. Ney, R. Haeb-Umbach, B.-H. Tran, M. Oerder. Improvements in beam search for 10000-word continuous speech recognition. ICASSP, 1992.

[7] K. Shim, M. Lee, I. Choi, Y. Boo, W. Sung. SVD-Softmax: fast softmax approximation on large vocabulary neural networks. NIPS, 2017.

[8] P. H. Chen, S. Si, S. Kumar, Y. Li, C.-J. Hsieh. Learning to screen for fast softmax inference on large vocabulary neural networks. ICLR, 2019.

[9] W. Tranheden, S. Ahmed, D. Dubhashi, J. Matthiesen, H. von Essen. FlashHead: efficient drop-in replacement for the classification head in language model inference. arXiv:2603.14591, 2026.

[10] Gemma Team. Gemma 4 technical report. arXiv:2607.02770, 2026.

[11] Weng et al. CORAL: learning consistent representations across multi-step training with lighter speculative drafter. arXiv:2502.16880, 2025.

[12] W. Zhao, T. Pan, X. Han et al. FR-Spec: accelerating large-vocabulary language models via frequency-ranked speculative sampling. ACL, 2025.

[13] VocabTrim: vocabulary pruning for efficient speculative decoding in LLMs. arXiv:2506.22694, 2025.

[14] E. Wijmans, B. Huval, A. Hertzberg, V. Koltun, P. Krähenbühl. Cut your losses in large-vocabulary language models. ICLR, 2025.

[15] S. Arora et al. Zoology: measuring and improving recall in efficient language models. arXiv:2312.04927, 2023.

[16] S. Jelassi, D. Brandfonbrener, S. M. Kakade, E. Malach. Repeat after me: Transformers are better than state space models at copying. arXiv:2402.01032, 2024.

[17] Z. Qin, S. Yang, Y. Zhong. Hierarchically gated recurrent neural network for sequence modeling. NeurIPS, 2023.